VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY

(An Autonomous Institute Affiliated to University of Mumbai)

Department of Computer Engineering

Project Report on

DeBug : A Decentralised Bug-Bounty Platform

In partial fulfilment of the Fourth Year, Bachelor of Engineering (B.E.) Degree in Computer Engineering at the University of Mumbai
Academic Year 2025 - 26

By

Rohit Shahi | D17C | Roll No. 58

Gopal Vanjarani | D17C | Roll No. 66

Gaurav Mahadeshwar | D17C | Roll No. 43

Umesh Tolani | D17C | Roll No. 64

Project Mentor

Mrs. Geocey Shejy

(A.Y. 2025 - 26)

VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY

(An Autonomous Institute Affiliated to University of Mumbai)

Department of Computer Engineering

Certificate

This is to certify that **Rohit Shahi (D17C - 58), Gopal Vanjarani (D17C - 66), Gaurav Mahadeshwar (D17C - 43) & Umesh Tolani (D17C - 64)** of Fourth Year Computer Engineering studying under the University of Mumbai have satisfactorily completed the project on **"DeBug : A Decentralised Bug-Bounty Platform"** as a part of their coursework of PROJECT - II for Semester - VIII under the guidance of their mentor **Mrs. Geocey Shejy** in the year 2025 - 26.

This thesis/dissertation/project report entitled **"DeBug : A Decentralised Bug-Bounty Platform"** by Rohit Shahi, Gopal Vanjarani, Gaurav Mahadeshwar & Umesh Tolani is approved for the degree of **Bachelor of Engineering** in **Computer Engineering.**

Programme Outcomes	Grade
PO1, PO2, PO3, PO4, PO5, PO6, PO7, PO8, PO9, PO10, PO11, PO12 PSO1 & PSO2	

Date: 27th April, 2026

Project Guide: Mrs. Geocey Shejy

Project Report Approval
For
B. E (Computer Engineering)

This project report entitled **"DeBug : A Decentralised Bug-Bounty Platform"** by **Rohit Shahi (D17C - 58), Gopal Vanjarani (D17C - 66), Gaurav Mahadeshwar (D17C - 43), & Umesh Tolani (D17C - 64)** is approved for the degree of **Bachelor of Engineering** in **Computer Engineering.**

Examiners

1. ……

(Internal Examiner name & sign)

2. ……

(External Examiner name & sign)

3. ……

(Head of Department)

4. ……

(Principal)

Date: 27th April, 2026

Place: Chembur, Mumbai

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Rohit Shahi - D17C / 58)

(Gopal Vanjarani - D17C / 66)

(Gaurav Mahadeshwar - D17C / 43)

(Umesh Tolani - D17C / 64)

Date : _____

Acknowledgement

We are thankful to our college Vivekanand Education Society's Institute of Technology for considering our project and extending help at all stages needed during our work of collecting information regarding the project.

It gives us immense pleasure to express our deep and sincere gratitude to Assistant Professor **Mrs. Geocey Shejy** for her kind help and valuable advice during the development of project synopsis and for her guidance and suggestions.

We are deeply indebted to the Head of the Computer Department, **Dr.(Mrs.) Nupur Giri** and our Principal, **Dr. (Mrs.) J.M. Nair**, for giving us this valuable opportunity to do this project.

We express our hearty thanks to them for their assistance without which it would have been difficult to finish this project synopsis and project review successfully.

We convey our deep sense of gratitude to all teaching and non-teaching staff for their constant encouragement, support and selfless help throughout the project work. It is a great pleasure to acknowledge the help and suggestion, which we received from the Department of Computer Engineering.

We wish to express our profound thanks to all those who helped us in gathering information about the project. Our families too have provided moral support and encouragement several times during the course of this work.

COURSE OUTCOMES FOR B.E. PROJECT

Learners will be able to,

Course Outcome	Description of the Course Outcome
CO1	Able to apply the relevant engineering concepts, knowledge and skills towards the project.
CO2	Able to identify, formulate and interpret the various relevant research papers and to determine the problem.
CO3	Able to apply the engineering concepts towards designing solutions for the problem.
CO4	Able to interpret the data and datasets to be utilized.
CO5	Able to create, select and apply appropriate technologies, techniques, resources and tools for the project.
CO6	Able to apply ethical, professional policies and principles towards societal, environmental, safety and cultural benefit.
CO7	Able to function effectively as an individual, and as a member of a team, allocating roles with clear lines of responsibility and accountability.
CO8	Able to write effective reports, design documents and make effective presentations.
CO9	Able to apply engineering and management principles to the project as a team member.
CO10	Able to apply the project domain knowledge to sharpen one’s competency.
CO11	Able to develop a professional, presentational, balanced and structured approach towards project development.
CO12	Able to adopt skills, languages, environment and platforms for creating innovative solutions for the project.

Index

List Of Figures

List Of Tables

Abstract

The fast increase in web applications, mobile applications, cloud platforms, and other forms of digital infrastructure has led to the expansion of the cyber-attack surfaces globally, which means that effective vulnerability disclosure and crowdsourcing of security efforts is essential. Existing bug bounty platforms offer valuable services to both parties, helping companies communicate their security vulnerabilities and get feedback from ethical hackers regarding ways to enhance their security posture. However, there are numerous issues associated with the current Web2 environment, including trust problems, delays in payments, lack of transparency, privacy breaches, as well as ineffective triaging procedures.

Thus, security researchers often find their reports ignored, experience reductions in payment amounts, or receive compensation later than expected. Simultaneously, companies are faced with challenges such as receiving multiple identical reports, plagiarized ones, and reports of low quality, which makes the job of a company's security team rather difficult. In this regard, it is possible to consider DeBug – A Decentralised Bug Bounty Platform as an innovative cybersecurity market designed to solve trust problems and automate various aspects of crowdsourced security.

In order to improve the quality of reports and decrease the amount of work during triaging, a system of AI-based analysis is implemented, where the LLMs serve as the basis for this system. Thus, the AI module is capable of conducting the semantic duplication detection, analyzing the exploit's description, identifying the method of exploitation, and estimating the severity level based on the CVSS standard. It allows for faster triaging, eliminates duplications of rewards, and streamlines vulnerability handling processes.

Other modules included in the platform include the encrypted report submission, decentralized storing via IPFS, wallet-based authentication, reputation management system, notifications, and dispute resolution system. The DeBug platform is built using the latest technologies, namely React.js/Next.js, Node.js, Solidity, and PostgreSQL/MongoDB. As a result, the combination of the blockchain and AI provides users with a powerful tool for conducting bug bounties with guaranteed security.

Chapter 1: Introduction

1.1 Introduction

Web applications, mobile devices, cloud-based services, and information technologies have seen tremendous growth, resulting in more cyberattacks across the world. Issues like API security risks, authentication problems, injections, and incorrect configurations pose serious cybersecurity threats. Many businesses and companies have taken advantage of the Bug Bounty program to enhance their cybersecurity by having ethical hackers and security researchers find out potential vulnerabilities in exchange for rewards.

Bug bounties have been successful in helping businesses protect themselves from cyber attacks; however, almost all of the available bug bounty platforms operate using a centralized Web2 platform to handle issues regarding submissions, rewards, and disputes between security researchers and the businesses. There is a problem of trust in centralized Bug Bounties since all aspects are managed by the organization or platforms. Researchers might be rewarded late or even face issues regarding rewards and transparency, while the organizations receive duplicate reports and other issues.

This paper recommends the adoption of DeBug - A Decentralized Bug Bounty Platform. This platform uses decentralized Web3 technology based on blockchain technology and smart contracts to overcome these challenges. Prior to the start of the program, organizations have to deposit the bounty rewards in secure escrow contracts on the blockchain. The smart contract then automatically distributes the funds when the reported issue is confirmed.

Also, the use of Generative Artificial Intelligence will be utilized for triaging the reports submitted. The GenAI engine helps in detecting duplicate submissions, generating summaries of exploits, and even in estimating the severity of issues following the CVSS guidelines.

1.2 Motivation

The main driving force behind DeBug is the key challenges that arise during modern cybersecurity practices. The efforts put forth by independent security researchers in identifying critical and even zero-day vulnerabilities do not always get proper attention and may lead to delays in reporting triages, being ignored altogether, or being silently patched by companies, leaving no reward to their rightful owners. Such actions create distrust and make many ethical hackers refrain from participating in bounty programs. From the organization's standpoint, the operation of any bounty program is associated with the receipt of tons of begging or even fake reports which may be duplicates of some previous ones or even plagiarism of somebody else's work. This creates additional work and distracts from the search for real vulnerabilities.

Finally, the traditional centralized bug bounty platforms lack transparency when it comes to payouts and disputes. In this respect, the development of DeBug was driven by the need to provide a transparent and intelligent bug bounty ecosystem based on the blockchain technology.

1.3 Problem Definition

There are some problems with the current bug bounty platforms. Most of the platforms use centralization, where the verification of reports, payments, and disputes are conducted only on one platform or company. These platforms lead to delayed payouts, rejection of reports, incorrect decisions on payments, and Silent Patching.

Also, organizations get duplicate reports, plagiarism reports, spam reports, and low-quality reports that increase the amount of work for the security department of the company. Manual verification and triaging processes take more time and effort.

Thus, the main problem that needs to be solved by the proposed platform is to create an environment without mutual trust between security researchers and companies. The platform should ensure fairness in payments using smart contracts. Moreover, there should be some mechanism for instant duplicate detection using AI.

1.4 Existing Systems

Currently, the bug bounty ecosystem is controlled by centralized intermediaries like HackerOne, Bugcrowd, and Intigriti. They act as the middlemen who facilitate bug reporting, communication, triage, and payment.

- **HackerOne**
  It is a widely adopted bug bounty intermediary that links researchers with businesses worldwide. It facilitates structured bug reporting programs but depends on manual validation and payment procedures.

- **Bugcrowd**
  It is a platform for cybersecurity assessments and bug reporting management for businesses. It includes triage services that are managed on its platform, but the bug reporting process is still centralized and manual.

- **Intigriti**
  It is another bug bounty platform with operations mostly centered in Europe. It focuses on private and public bug bounty programs, supporting coordinated disclosure, but the process is centralized.

In such platforms, enterprises deposit regular fiat money in their accounts through bank transactions. Manual triage teams check incoming PDF or Markdown reports for vulnerabilities' exploitability and severity. Furthermore, the platform also charges a heavy service fee, usually more than 20%, from the enterprise's bug bounty fund. Although such platforms are efficient, they are often faced with problems like delays, lack of transparency, high operational costs, and dependency on intermediaries.

1.5 Lacuna in the Existing Systems

Despite the advantages provided by the currently available bug bounty programs, there are a number of disadvantages inherent to their nature that prevent further development of the bug bounty approach.

Single Point of Failure/Control: Being centralized, the platforms control the entire process, including the data exchange, the researcher's reputation record, and the payment process. Therefore, if a researcher gets blocked, he/she faces problems with his/her reputation and rewards.

Fiat Frictions: Payouts made through the bank accounts require the researchers to have an account at a bank in the country where the program is based. As a result, researchers from other countries will face significant restrictions.

Asymmetric Judgement: The organizations unilaterally decide whether or not the report is a duplicate. This approach may create problems related to the fairness of the decision and researchers' morale.

Triage Latency: Human triage of reports takes much time during which vulnerability severity verification and payout processing are being done.

Such lacunas highlight the need for a transparent and decentralized Bug Bounty Ecosystem.

1.6 Relevance of the Project

The suggested system is quite pertinent in terms of current cybersecurity trends since companies need a more effective, transparent, and scalable approach to vulnerability management. In light of the growing number of cyber attacks, existing bug bounty systems do not deliver timely rewards, borderless and bank-less payments, and rapid handling of submitted reports.

By merging the two aforementioned areas, DeBug creates an innovative vulnerability disclosure mechanism for the digital age. Researchers receive irrefutable, blockchain-based evidence of their submissions, as well as instant crypto rewards, bypassing any geographical limitations or delays in bank transfers.

On the other hand, enterprises get access to artificial intelligence-based triage systems that can automatically analyze reports' content, find duplicates, summarize exploits' descriptions, and estimate their severity in a matter of seconds. This dramatically cuts down on the resources needed to sustain constant bug bounties.

Consequently, DeBug is highly pertinent as it leverages decentralization, automation, and global availability to improve current bug bounty networks.

Chapter 2: Literature Survey

A. Brief Overview of Literature Survey

The idea of bridging the gap between Blockchain Smart Contracts with automated security validation software is an extensively researched area of modern computer science. The major chunk of literature available on the topic deals with either finding vulnerabilities within the smart contract itself, or using NLP based models to categorize technical documents or help with support tickets.

In conducting literature surveys for this project, research papers and journals were extensively examined through IEEE, Springer, ACM and other renowned publishers within the domain from 2021-2025. The research covered topics like Decentralized Escrow Solutions, AI based CVE analysis, Vulnerability report classification, and Web3 identity architecture.

The papers were helpful in determining that blockchain technology can be used to carry out financial transactions without the need for trust, artificial intelligence can be used for automating technical analysis, and how decentralized identities can be developed for improved privacy and identification.

B. Related Works

2.1 Research Papers Referred

1. **Hoffman et al., "Bountychain: A Decentralized Bug Bounty Platform," IEEE, 2021**

   Abstract:

   This paper proposes the creation of a platform for bug bounties with the use of blockchains and smart contracts. With the use of these technologies, rewards can be automatically allocated while keeping record of vulnerabilities that are submitted.

   Inference:

   For DeBug, this paper provides the basis for the implementation of blockchain-based escrow contracts and decentralized storage of reports.

2. **"Automating Cybersecurity Triage via Large Language Models," IEEE, 2024**

   Abstract:

   In this research, we looked at the application of Transformers for parsing the payload sent by users. We found out that large language models perform better than abstract syntax tree methods in detecting syntax structures of exploits like XSS and SQL injection attacks.

   Inference:

   Our project leverages state-of-the-art models like Gemini 1.5 Flash trained on code repositories. Early research demonstrated that large language models could generate inaccurate severity scores. For this reason, our solution employs JSON templates aligned with CVSS standards.

3. **"Trustless Escrow Protocols using Solidity Smart Contracts," Springer, 2023**

   Abstract:

   The study outlined various frameworks in which numerous participants would mathematically lock up collateral using cryptographic statements on the Ethereum platform without relying on arbitrators.

   Inference:

   In the case of DeBug, it ensures that there are high financial incentives for research. It uses the approveAndPay method, whereby mathematical escrow is combined with human verification of bugs.

4. **J. Kumar et al., "AI-Assisted Vulnerability Report Classification using NLP," IEEE Access, 2022**

   Abstract:

   In this study, Natural Language Processing was applied to categorize technical vulnerability reports, detect duplicates, and rank the reports according to their severity level.

   Inference:

   With regards to DeBug, this research will aid in detecting duplicates, automated categorization, and prioritization of the reported vulnerabilities.

5. **R. Sharma et al., "Smart Contract Escrow Systems for Secure Payments," Springer, 2021**

   Abstract:

   The above discussion focused on escrowed smart contracts that automate payment once certain conditions are met. This system helps eliminate cases of fraud, delays, and reliance on third parties.

   Inference:

   In DeBug, the application of this theory will make it possible to pay bounties automatically.

6. **Lee et al., "Large Language Models for CVE Analysis and Security Summarization," IEEE, 2024**

   Abstract:

   The current study explores the application of Large Language Models in security-related tasks to analyze descriptions of vulnerabilities, summarize exploits, and support severity analysis.

   Inference:

   With regards to DeBug, this is an excellent application of AI to help in faster report triaging, exploit summarization, and vulnerability management.

2.2 Patent Search

2.2.1 US10860748B2: Automatic Detection of Offensive Textual Content Using Machine Learning (Reference Concept for AI Classification Systems)

Abstract:

The current Patent discloses a machine learning system which is used to classify text based on predetermined categories. The system uses the neural network training methods that analyze text and identify linguistic features in the process of automated detection of any offensive or suspicious text. This invention is relevant for DeBug where an equivalent solution can be utilized for automated analysis of vulnerability reports, duplicate identification, report quality classification, and spam filtering.

US Patent Link: https://patft.uspto.gov/patft/search/

2.2.2 US20230186957A1: System for Detecting Health Misinformation using Natural Language Processing

Abstract:

In this patent, we have an NLP (Natural Language Processing) system that is used for identifying deceptive digital information by doing semantic analysis on the textual content. This system utilizes artificial intelligence to produce output based on confidence level for classifying information present in online content. This methodology can be applied to DeBug for identifying semantic duplicates, vulnerabilities, and intent behind exploits.

US Patent Link: https://patft.uspto.gov/patft/search/

2.2.3 EP4012671A1: Machine Learning Based Harmful Content Moderation Platform

Abstract:

The following European patent provides an example of a machine learning-based framework for assessing and moderating dangerous online content. This system minimizes human effort by targeting potentially problematic information while filtering out redundant or worthless inputs. In the case of DeBug, this approach could be adopted to target important vulnerability disclosures, minimize triaging delays, and automatically detect plagiarized or low-quality content.

US Patent Link: https://worldwide.espacenet.com/

2.2.4 US20190392291A1: Smart Contract Escrow Payment System using Blockchain

Abstract:

The Patent is related to a blockchain-based escrow mechanism involving locking of digital money through smart contracts and its release only upon satisfaction of certain conditions. This eliminates reliance on third parties for any transaction. In relation to DeBug, this is relevant to the escrow process associated with bounties. Funds are initially locked by the organizations before embarking on the bounty program.

US Patent Link: https://patft.uspto.gov/patft/search/

2.2.5 EP3472091A1: Decentralized Identity Verification Framework

Abstract:

This patent presents an identity system that allows authentication in a safe way without divulging all the user's personal details. It employs cryptographic proof along with identity verification based on wallets. For DeBug, this concept can be useful when implementing the logging-in process in wallets and also providing an optional zero-knowledge identity verification service.

US Patent Link: https://worldwide.espacenet.com/

2.3 Inference Drawn

From the comprehensive review of literature and patents, the following key inferences were drawn:

- Blockchain technology provides secure, transparent, and immutable financial transactions, making it suitable for the automated payout processes in bug bounties.
- LLM models and Transformer architectures are great in processing vulnerability reports, detecting duplicates, summarizing exploits, and assisting with prioritization.
- To prevent hallucinations and ensure consistency in the AI process, AI output should be directed by predefined schemas and CVSS metrics.
- Real-time triaging requires a well-engineered backend: optimized API services, quick retrieval of data, effective pipelines, and real-time inference engines.
- Most current solutions focus on solving one problem at a time – whether it is payouts, moderation, or classification, whereas combining all three would be optimal.

2.4 Comparison with the existing systems

Sr. No.	Feature	Existing Systems (HackerOne / Bugcrowd / Intigriti)	Proposed System (DeBug)
1.	Architecture	Run on centralized Web2 architecture in which the platform is responsible for managing submissions, reporting process, payments, and dispute resolution.	Utilizes decentralized Web3 architecture, wherein key transactions such as payments and trusted operations occur on blockchain via smart contracts.
2.	Reward Payment System	Payouts are handled manually using fiat banking systems, which can result in payment delays, transaction costs, and geographical limitations.	Entities use smart contract escrow to hold bounty money and facilitate quick crypto transfers post-approval.
3.	Transparency & Trust	Researchers have to rely on the integrity of the platform and company for unbiased decisions on duplicates, severity, and payouts.	Blockchain transactions offer evidence of locked funds, approved bounty payments, and transferred payments.
4.	Report Triage Process	The security team evaluates reports manually, resulting in delays in case of duplicate, spam, or poor-quality reports.	AI-driven triage system carries out tasks including duplicate removal, exploit summarization, and severity rating to alleviate workload.
5.	Global Accessibility	Reporting may be hindered by banking systems, currency exchange rates, or geographical restrictions.	Participation via wallet allows borderless research participation from anywhere in the world without relying on conventional banking.
6.	Data Storage & Security	Reports are centrally stored in vulnerability databases, raising concerns about unauthorized access or single point of failure.	Confidential data may be stored securely using a decentralized system such as IPFS with restricted access.
7.	Operational Cost	High service charges are applied, and extensive human resources are needed to manage reports.	Minimized dependence on intermediaries results in lowered expenses.

Table no. 1 – Comparison of existing systems and proposed area of work

Chapter 3: Requirement Gathering for the Proposed System

3.1 Introduction to requirement gathering

The process of requirement gathering for DeBug involved a systematic analysis of the following: Existing bug bounty platforms, cybersecurity processes, payment using blockchain technology, and automation based on Artificial Intelligence. The purpose of this research is to determine real-life problems faced by enterprises and cybersecurity professionals in vulnerability disclosure.

The process included the analysis of traditional bug bounty platforms HackerOne, Bugcrowd, and Intigriti to determine the inefficiencies of the platforms such as delayed payments, manual triage, duplicity of reports, and lack of transparency and centralized authority. In order to discover the improvements offered by blockchain technology, research papers, and patents along with current Web3 solutions were reviewed in order to learn about smart contracts, decentralized storage, and identity wallets.

Finally, Large Language Models' capabilities have been studied in order to determine the use of AI for automatic triage including categorizing and deduplication of reports, exploits' summary, and threat estimations. This information is used to gather the requirements for DeBug.

According to requirement gathering results, DeBug is a hybrid solution that combines blockchain technologies and AI for an efficient bug bounty program management.

3.2 Functional Requirements

- **Web3 Authentication Implementation:** The solution will have to connect to injected browser wallets (for example MetaMask) with Ethers.js and will use cryptographic key pairs for authentication purposes.
- **Smart Contract Creation:** Organizations must be able to create isolated BountyEscrow Smart Contracts for certain bounties.
- **Cryptographic Escrow Logic:** The contract will require locking of the ETH at the moment of creation and implementation of the approveAndPay function which is only executable by the organizational owner.
- **Safe and Reliable Payload Sending:** It will need an advanced contextual editor (which supports Markdown), capable of sending Proof of Concept exploits to the backend database.
- **Automatic Exploits LLM Analysis:** The back-end side will need to call the Google Gemini Generation API on-demand to analyze the PoC exploit text to create a structured JSON.
- **Plagiarism & Duplicate Detection:** The system will check the structure of the exploit report against previous entries in the database.

3.3 Non-Functional Requirements

- **Performance:** The system shall respond fast for report submission and AI classification.
- **Latency in Verifying Results:** The process of executing the AI triage should not exceed a tolerance time of 15 seconds.
- **Immutability and Visibility:** The financial routing code (the EVM bytecode) should be immutable and visible through the blockchain explorer.
- **UI Design Aesthetics:** The UI design of the client application should be modern and dynamic, i.e., glassmorphic, making use of WebGL capabilities.
- **Gas Optimization:** The Solidity contracts should be optimized such that any deploy/paying transaction takes up the least amount of gas usage possible (through the use of calldata, variable packings, and custom errors).

3.4 Hardware, Software , Technology and tools utilized

- Frontend Development:
  React.js, Vite, TailwindCSS equivalent custom utility styles, Framer Motion (Animations), React-Three-Fiber (3D WebGL Canvas).
- Backend Application:
  Node.js (v20+), Express.js framework.
- Database & Identity Layer:
  Supabase (PostgreSQL implementation).
- Blockchain Protocol Layer:
  Hardhat (Local Ethereum Node), Solidity (v0.8.19), Ethers.js (v6).
- Artificial Intelligence:
  Google Gemini 1.5 Flash Large Language Model via @google/generative-ai SDK.

3.5 Constraints

- **Gas Fees Variability:** Since the platform executes on Ethereum (or L2s such as Polygon), varying network congestion can impact the fiat equivalent cost of deploying a bounty escrow.
- **Zero-Day Confidentiality:** Due to the public ledger nature of blockchains, the sensitive vulnerability data (PoC payloads) cannot be stored directly on-chain; hence they are held centrally in the Supabase DB while only the financial mechanics reside immutably on-chain.
- **GenAI Context Window Limitations:** Exceptionally massive crash-dump files or thousands of lines of source code attached to a report may exceed the LLM API token limit, requiring manual evaluation.

Chapter 4: Proposed Design

4.1 Block Diagram of the proposed system

DeBug is expected to be based on a hybrid architecture comprising centralized application services combined with the decentralized blockchain technology. Four key layers are involved, which include Frontend Layer, Backend Layer, Web3 Layer, and Persistence Layer. They all play crucial roles in ensuring authentication, reporting vulnerabilities, triage by artificial intelligence, and payments to hackers.

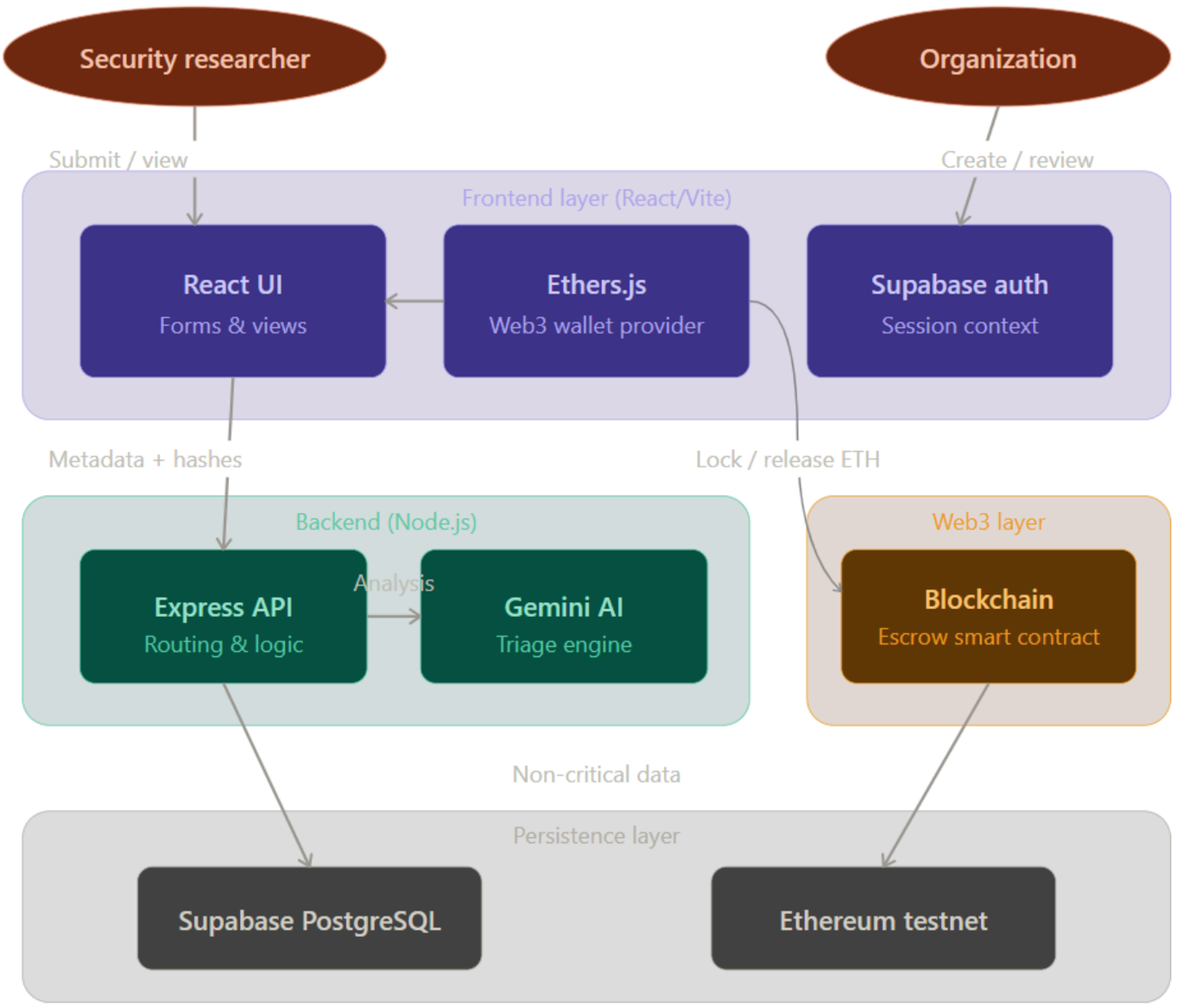

Fig 4.1.1 – Block Diagram

Explanation of System Architecture:

- User Entities – The main entities that use the platform are Security Researchers and Organizations. The first group submits vulnerability reports, whereas the second one creates bounties, reviews reports and approves rewards.

- Frontend Layer (React/Vite) – This layer contains all the interfaces for interacting with the system. Here, React dashboards, report form, bounty management interfaces, and wallet connectivity using Ethers.js reside. It allows users to easily communicate with blockchain.

- Authentication Layer – User authentication can be done by Supabase authentication, and additionally, by using wallet. This way, it ensures the security of interactions.

- Backend Layer (Node.js/Express) – This layer takes care of handling API calls, validating the information provided by users, executing the business logic and storing metadata, and providing the connection between the frontend, Gemini AI, Database, and Blockchain layer.

- AI Triage Engine (Gemini AI) – In this module, reports are analyzed to determine if they contain duplicate information, provide the summary of exploits, and estimate the severity level of vulnerabilities. Organizations will be able to reduce the triaging time.

- Web3 Layer (Blockchain) – This layer contains a smart contract system, which will manage the bounty program, including locking the funds of rewards until researchers are approved to receive them.

- Persistence Layer – PostgreSQL via Supabase is utilized to store data pertaining to users, bounties, submissions, notifications, and logging purposes. Transactions related to blockchain will be stored on the Ethereum testnet.

Overall Workflow – Researchers generate reports on the front end. The backend processes them and analyzes them using AI. Organizations then examine the findings and make final payments through smart contracts. In conclusion, the presented architecture efficiently integrates the efficiency of centralized databases with the advantages of decentralized solutions.

4.2 Modular design of the system

The DeBug system consists of modules that complement one another without depending on each other. Each module has its designated function, namely user logging and validation, bounty creation, vulnerability submission, AI-based triaging, payment by blockchain technology, and secure data storage. The modular approach used in development helps to structure the platform and simplify the process of coding.

This modularity ensures that maintenance is easy, and development proceeds smoothly. New functionality can be introduced with minor modifications, making the platform flexible, reliable, and effectively managed.

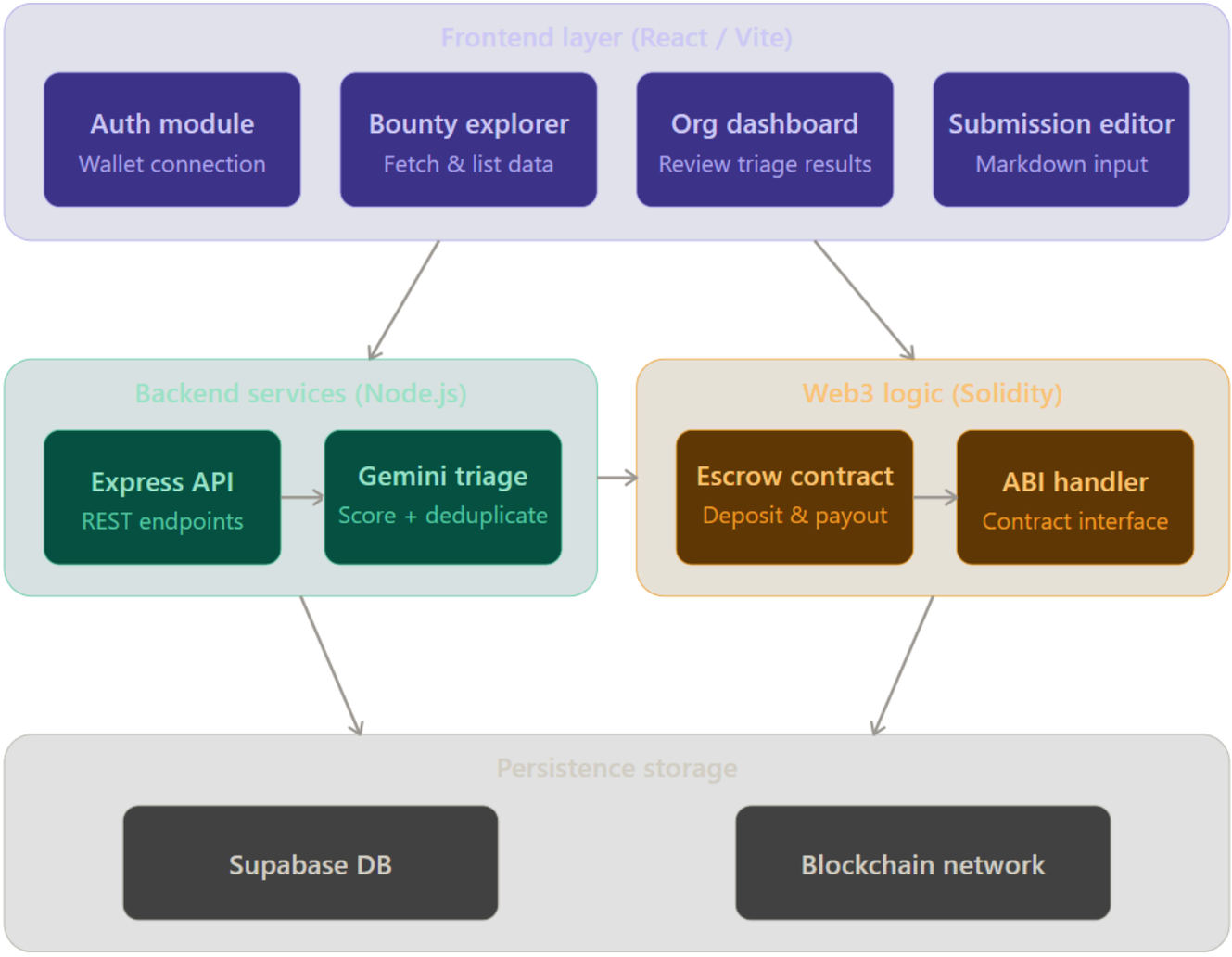

Fig 4.2.2 – Modular Diagram

Modular Design of the System Explanation:
The suggested architecture of the DeBug platform includes several different modules that contribute to the efficient implementation of bug bounties.

- **Authentication Module -** In contrast to conventional authentication based on usernames and passwords, in this module authentication of the user's identity is achieved using his/her cryptocurrency wallet address.

- **Mission Control (Bounty Creation) Module -** This module is intended for organizations to create and manage their own bug bounty program. In particular, an organization defines target domains, scope rules, rewards, and releases new Bounty Escrow smart contracts.

- **Researcher Dashboard Module -** This module provides security researchers with access to information about bounty programs, reported vulnerabilities, levels of severity, and rewards. In addition, it implements a system of researcher reputation based on Levels and XP gained from successful reports.

- **AI Intermediary Module -** It represents an advanced analytical component of the platform. The module processes incoming reports, identifies duplicates by matching against historical databases of reports, and calculates confidence scores of reported vulnerabilities.

- **Blockchain Payment Module -** The module implements blockchain escrow contracts to process payments of rewards.

Combining these modules results in a modular system where authentication, bounty creation, artificial intelligence-based prioritization of requests, reputation management, and payment distributions operate in a cohesive manner.

4.3 Detailed Design

The Smart Contract is strictly designed as a highly optimized state machine containing the following properties:

- address public organization; (Deployer)
- uint public bountyAmount;
- bool public isActive; The contract implements restrictive Modifiers (e.g., onlyOrganization) ensuring exclusively the fund depositor can release the ETH capital via the target approveAndPay function.

On the Backend Database side, the schema utilizes robust relations, primarily linking reports directly to an overarching bounties ID. Real-time updates push notifications when the status enum pivots from "submitted" to "accepted" or "rejected".

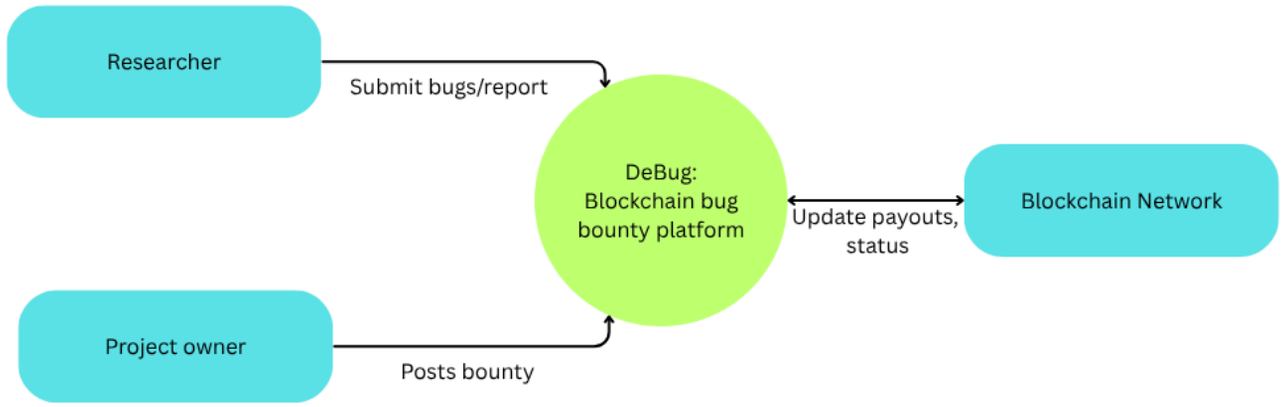

Fig 4.3.1 – DFD level 0

The following DFD Level 0 represents the high-level data flow of the proposed DeBug platform. It shows how the main external entities interact with the central bug bounty management system and the blockchain network. The diagram focuses on the basic movement of information such as bounty creation, vulnerability report submission, and payout status updates.

Under this scheme, the researcher submits bug reports using the DeBug software, and the project owner or organization establishes and manages the bounties. The DeBug software takes care of the above processes and communicates with the Blockchain Network regarding the escrow payments, transaction logs, and rewards. Therefore, the figure gives an easy-to-understand illustration of the key interactions within the system.

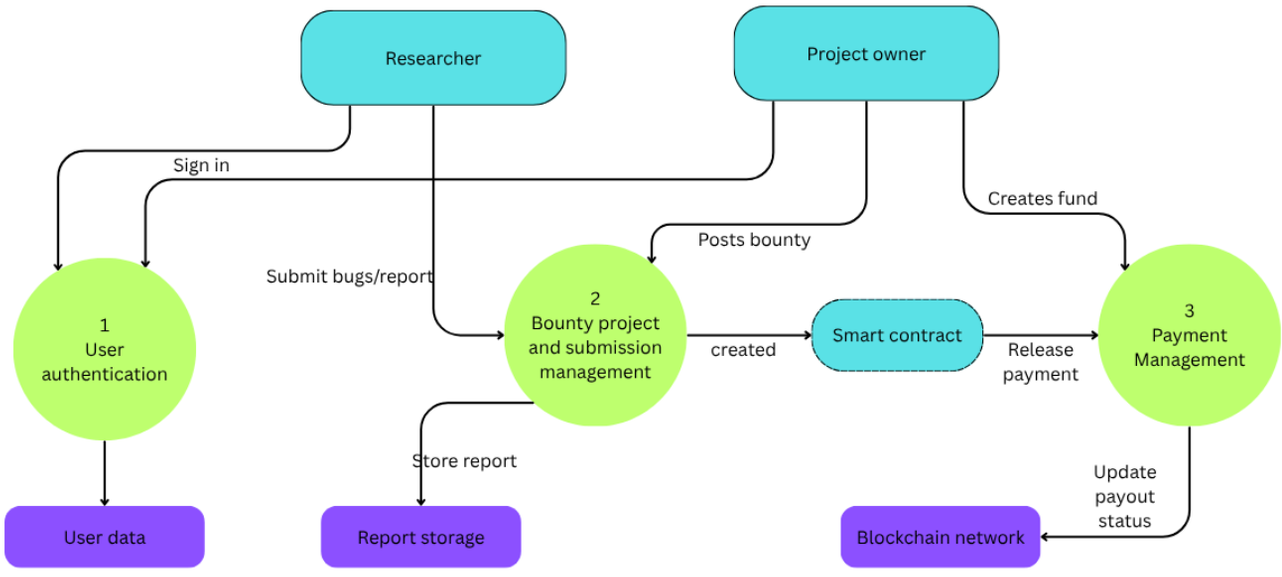

Fig 4.3.2 – DFD level 1

The following DFD Level 1 represents a more detailed view of the internal data flow of the proposed DeBug platform. It breaks the main system into key functional processes such as user authentication, bounty and submission management, and payment management.

The diagram also shows the interaction of these processes with storage components and blockchain services.

In this model, the Researcher and Project Owner first complete authentication through the user verification process.

After login, the project owner can post bounty programs, while researchers submit vulnerability reports. Reports are stored in the report database and bounty details are managed by the central system.

The payment process occurs through smart contracts communicating with the blockchain to dispense rewards and track payouts. To summarize, the flowchart provides the operations of the platform.

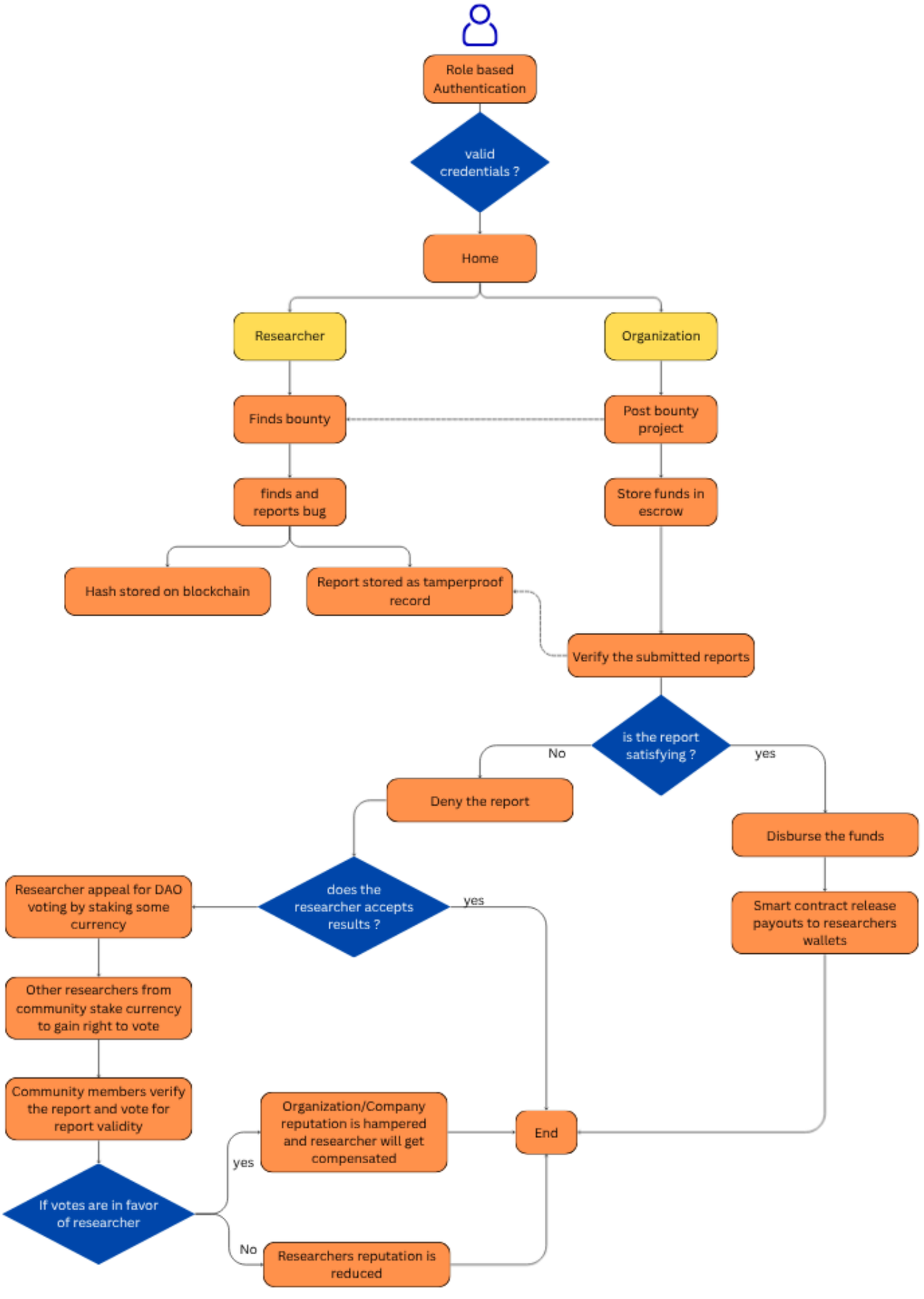

Fig 4.3.3 – Detailed Design flow chart

Flow Explanation :

- The user accesses the platform and completes role-based authorization as Researcher or Organization through their wallet.
- On successful authorization, the user is directed to the Home Dashboard.
- Researcher Flow:
  - Explore the bounty programs available.
  - Find the vulnerability in the target system.
  - Make a bug report with proof-of-concept data.
  - The report is saved safely and evidence of submission is documented.
- Organization Flow:
  - Create and launch a bounty program.
  - Set the parameters and terms of engagement.
  - Deposit funds in smart contracts escrow account.
- The submitted bug reports are analyzed by AI triage to check for duplicates and assess severity.
- Check if the report qualifies the bounty.
- On being Approved:
  - The smart contract sends the reward money to the researcher's wallet.
- On being Rejected:
  - The researcher can raise an appeal for the community/DAO consideration.
- DAO Consideration:
  - For votes in favor of the researcher -> Reward distributed and organizational reputation reduced.
  - Against appeal of the researcher -> Researcher reputation reduced.
- The process completes successfully.

4.4 Project Scheduling & Tracking using Timeline / Gantt Chart

The Gantt chart indicates the well-planned schedule adopted during the creation of the DeBug project through several stages.

- Initial Phase (Week 1 – Week 2): The topic, scope, literature, and synopsis were defined. The constraints of the current bug bounty platforms were analyzed, and the project concept was formed.
- Planning Phase (Week 3): Requirement collection, ideation, process definition, DFDs, and system design were done.
- Development Phase (Week 4 – Week 7): The backend APIs, database connectivity, wallet authorization, blockchain smart contracts, artificial intelligence-based triage algorithms, and frontend user interface dashboards were implemented.
- Testing Phase (Week 7 – Week 8): System functionality testing, bugs correction, smart contract verification, and performance analysis were performed.
- Final Phase (Week 9): System evaluation, report writing, documentation, and project submission were done.

Therefore, the Gantt chart facilitated better project management.

Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9
Topic Discussion & Scope Finalization									
Literature Review (Full Draft)									
Synopsis Submission (Formal Hand-in)									
Requirements Brainstorming (DFD/Flowchart)									
Review 1									
Adding the Backend Logic									
Building of UI/Frontend									
Testing									
Review 2									

Fig 4.4.1 – Gantt Chart

Chapter 5: Implementation of the Proposed System

5.1. Methodology employed for development

DeBug has been developed in a structured manner, which allows for efficiency in implementing all modules of the application. The application has been developed following an iterative process in which modules are implemented and tested at different stages.

- **Requirement Analysis:** An analysis of existing bug bounty websites revealed the following difficulties: delay in payments, multiple submissions of the same report, and lack of transparency. Accordingly, system requirements were determined.
- **System Design:** A hybrid design for the system, comprising the frontend, backend, blockchain, database, and artificial intelligence parts was devised. A diagram showing workflows and interactions between different modules was drawn up before the development process started.
- **Frontend Development:** Dashboards, bounty cards, report forms, and trackers were developed for researchers and organizations using React.js/Vite.
- **Backend Development:** REST APIs for authentication, bounty management, reporting, notification, and interaction between modules were created using Node.js/Express.js.
- **Blockchain Development:** Escrow mechanisms, fund lockers, and payout schemes were implemented using smart contracts written in Solidity.
- **Artificial Intelligence Integration:** Report duplication, summarization, and severity estimation functions were performed using Google Gemini API.
- **Database Development:** The storage of users' information, bounties, reports, and logs was performed using Supabase (PostgreSQL).
- **Testing and Deployment:** Individual testing of all modules and their integration into the complete system was carried out.

It is, therefore, through such a methodology that involves planning, module creation, testing, and eventual integration that the DeBug solution was effectively developed as a scalable, secure, and efficient decentralized bug bounty platform.

5.2 Algorithms and flowcharts for the respective modules developed

Algorithm 1 – Dynamic Escrow Generation

Step 1: The organization binds its wallet and signs the MetaMask digital signature to enable secure authorization.

Step 2: In the frontend, the BOUNTY_ESCROW_ABI together with the compiled code is used to create the ContractFactory.

Step 3: The organization deploys the contract by setting the necessary amount of transaction value in ETH according to the chosen bounty tier.

Step 4: Once the deployment is confirmed, the generated smart contract address is saved in the organizational bounty records in Supabase.

The algorithm guarantees that the funds allocated for rewards are in escrow before the launch of the bounty campaign.

Algorithm 2 – Implementation of TF-IDF and Semantic Duplicate Filtering using GenAI

Step 1: The backend gets the input vulnerability report submission as a text string.

Step 2: The database looks up past reported vulnerabilities with the corresponding bounty_id.

Step 3: The past vulnerabilities' reports are transformed into a comparison array.

Step 4: Instructional prompts are passed through the Google Gemini API to calculate semantic similarity of the report to past reports.

Step 5: In case of a higher than 80% similarity rate, the current report is considered duplicated and follows the reject branch.

Step 6: Otherwise, if there is no duplication detected, the AI engine evaluates exploitation details and gives a CVSS scoring result.

Step 7: The obtained report's features are saved to a JSON file.

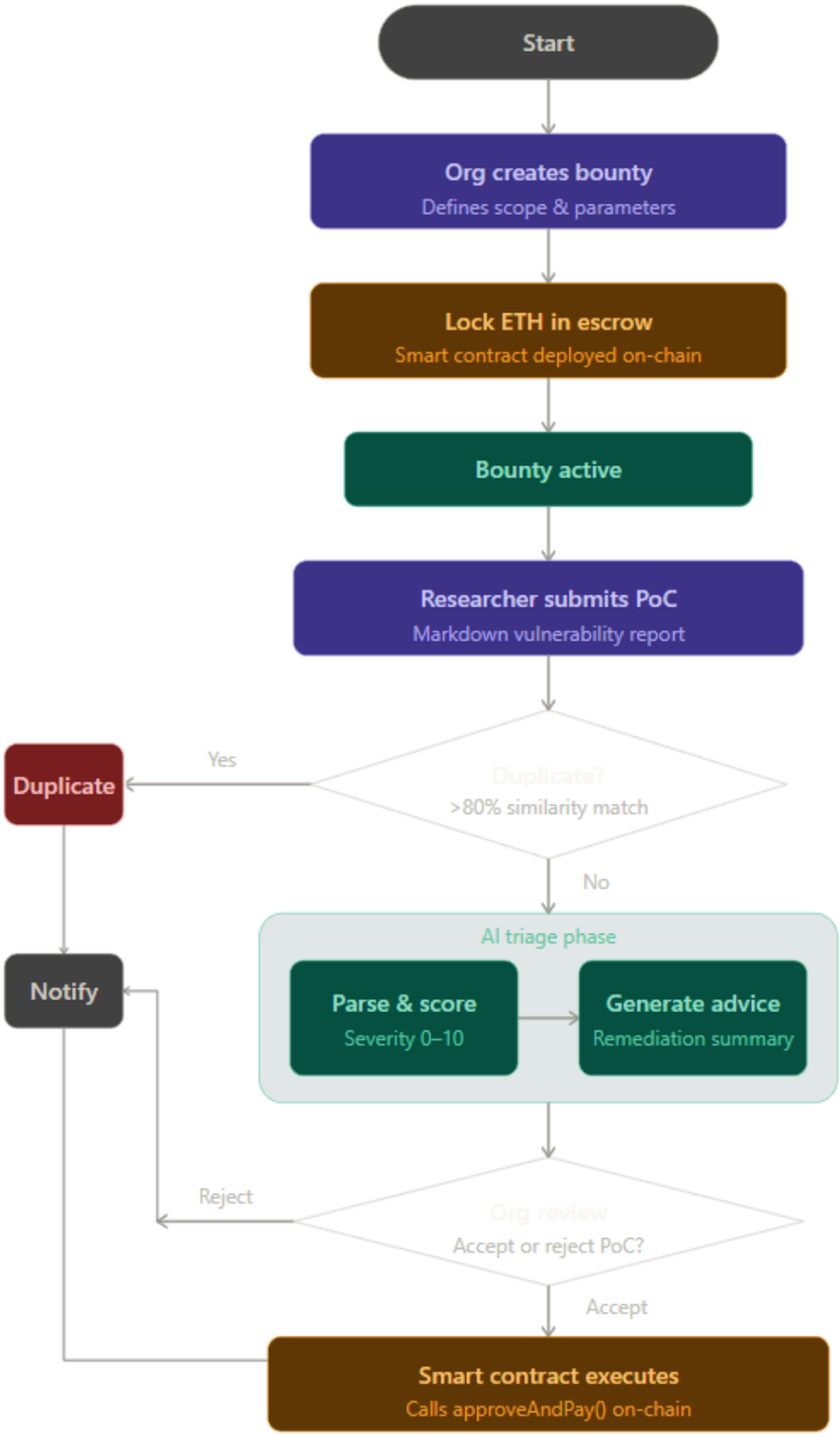

Fig 5.2.1 – Flowchart

Explanation :

The DeBug flowchart demonstrates the full operational algorithm for the platform from creating bounties to receiving reward payments securely.

- The algorithm starts with the step when the Organization initiates bug bounty programs, specifying the scope, rewards, and submitting criteria. As soon as the bounty is set up, the Organization should lock in escrow the necessary amount of ETH through the smart contract. With successful deployment of the smart contract, the bounty program will be available to use.

- Next, the Researcher examines the target application, writes down the PoC vulnerability report and submits it through the editor on the platform. Once submitted, the system checks whether there was a similar report before, comparing it to other vulnerability entries. When the similarity score exceeds the established level, the report is recognized as duplicate, and the message about that is sent to the researcher.

- In case the report is unique, its next stage is AI-powered triaging. The AI engine processes the PoC report, identifies crucial exploit data, estimates the severity of a vulnerability on a scale from 0 to 10, and formulates recommendations to address it.

- After analysis by AI, the report goes through the Organization Review Module, where the organization validates the report and determines whether to accept it or not. In case of rejection, the researcher will receive a notification about the result of the process.

- In case the report gets accepted, the smart contract will execute the approveAndPay() method. The amount of reward, which was earlier locked for this particular bounty program, is then safely transferred into the wallet of the researcher.

Therefore, it can be observed that the algorithm provides for effective bounty program management, automatic triage, reduction of duplicates, and trusted payouts.

5.3 Datasets source and utilization

In contrast to regular Machine Learning models requiring extensive static data training sets, DeBug is implemented using the Large Language Model inference in Zero-Shot prompting mode. As the embedded AI engine is prompt-based and doesn't require custom model training, there was no necessity in developing a large dataset for this purpose.

At the same time, to test and validate the triaging capabilities of the AI triage module, a micro-dataset of fifty classical CVE vulnerability reports was created. Such vulnerabilities could be of various types including Log4j vulnerabilities, SQL injection instances, DOM-based XSS attacks, authentication issues, and standard web exploits. The CVE database was sourced from public information in the MITRE Corporation CVE Database.

The mentioned dataset was extensively used throughout the testing process to fine-tune prompt instructions and adjust other settings such as temperature and back-end logic.

Main purposes for which the dataset was used are listed below:

- Testing duplication and semantic similarity capabilities;
- Verifying vulnerability severity mapping via CVSS metrics;
- Keeping output data strictly JSON format compliant;
- Enhancing exploit descriptions and recommendations.

Therefore, even though DeBug does not rely on standard training datasets, the use of CVE samples proved vital in enhancing the accuracy of the artificial intelligence triaging tool.

Chapter 6: Testing of the Proposed System

6.1 . Introduction to testing

Tests will be indispensable for the Debug project, as they help validate the operation of its components and ensure that it functions properly. The reason is that the proposed decentralized platform is based on several modern technologies such as Web3 smart contracts, which use Solidity code, generative AI (Gemini AI), and Supabase database management technology.

In this regard, the tests should address important issues related to the Web3 wallet verification, smart contracts' operation, and the transactional nature of the escrow mechanism. Also, they should cover the AI-based bug triaging and the synchronization of the user profile analytics with the blockchain and off-chain databases.

In addition to these features, testing can identify critical vulnerabilities, inconsistencies in integration, and performance bottlenecks of the frontend (React framework), backend (Node.js), and the whole blockchain part (written in Hardhat and Ethereum network).

6.2. Types of tests Considered

Various tests were done to make sure the project is robust, secure, and reliable:

1. Unit Testing Unit testing -

It was done on standalone components/functions. In this test, there were tests done on individual functions such as those in the Solidity smart contracts (deposit, releaseFunds, and access controls). The others tested were the fallback database calculations and the Gemini prompt instructions. Each component was tested heavily before interacting with other systems.

2. Integration Testing Integration testing -

It made sure that different components interact properly with each other. Some of the integration tests were done between the frontend (React) and the Ethereum blockchain using Ethers.js, the backend and the external Gemini API, and finally the frontend interacting with Supabase. This test ensures that all data flows well when changing the status from "submitted" to "accepted."

3. System Testing System testing -

It ensured the whole decentralized system works. This is where all components of the project work in a continuous process. Some of the processes include connecting a wallet, depositing escrow funds for a bounty, submitting a vulnerability, doing AI triage, and finally the on-chain payout.

4. Performance Testing Performance testing -

It was conducted to test the API responsiveness and the efficiency of the smart contracts used. The performance testing involved the following: latency of the GenAI response while processing long markdown files for PoC and transaction confirmation time on the local blockchain node.

5. Security Testing -

Considering that the platform was situated in the area of cybersecurity, extensive security testing was done on the smart contracts. Security measures included the mitigation of potential attack vectors including Re-Entrancy attacks, use of role-based access control (only allowing access from the org wallet for fund release purposes), and environment isolation.

6. Usability Testing -

Usability testing mainly involved assessing how effective the user interface was in abstracting the complexities of "Web3". Usability testing was done considering the ease of navigating through the application and its dashboard's responsiveness and transaction statuses.

6.3 Various test case scenarios considered

The following test cases were designed to validate key functionalities of the system:

Test Case ID	Module	Test Scenario	Expected Result	Status
TC01	Authentication	Connect via MetaMask wallet	User signature verified, profile loaded successfully	Passed
TC02	Bounty Creation	Deploy new Escrow Contract	Contract deployed with assigned ETH balance	Passed
TC03	Report Submission	Submit valid vulnerability report	Data safely stored in DB mapped to correct Bounty ID	Passed
TC04	AI Triage	Submit duplicate / invalid bug	AI flags report, generates low severity score	Passed
TC05	AI Triage	Submit critical zero-day bug	AI confirms validity, sets High impact severity	Passed
TC06	Payout Processing	Organization accepts report	Smart contract transfers assigned ETH to Researcher	Passed
TC07	Dashboard Analytics	Refresh Profile post-payout	Reports Accepted and Total Earned stats increment properly	Passed

Test Case ID	Module	Test Scenario	Expected Result	Status
TC08	Smart Contract Security	Unknown user tries withdrawing	Transaction forcefully reverted via Access Control	Passed
TC09	UI Rendering	Load Markdown Proof-of-Concept	Code blocks and text styles render beautifully	Passed
TC10	Error Handling	Reject user MetaMask transaction	Graceful error prompt without crashing application	Passed

Table no. 2 – Test Cases

6.4 Inference drawn from the test cases

From the testing results, Debug appears to be quite dependable in its operations across essential features. The program ensures reliable coordination of off-chain artificial intelligence analysis with on-chain financial transactions, offering seamless trustless use to users. Integration of Web3 wallets into the platform abstracts friction perfectly.

The triage system powered by artificial intelligence works as an extremely effective sieve, identifying technical ramifications of the vulnerabilities reported and efficiently lowering the burden of triaging the vulnerability reports manually for organization security teams. The smart contract escrow system works flawlessly, effectively addressing any trust-related concerns by securing the funds mathematically, and researchers get the precise amount of payment once their reports are accepted.

Metrics and dashboard analytics appear to be seamlessly synced with database triggers, leading to an exciting leaderboard and reputation system.

The system appears to be extremely reliable and secure, offering seamless usability. In the future, optimization of the smart contract gas consumption matrix, multi-chain support, and Gemini prompts for cryptography vulnerabilities could be improved.

Chapter 7: Results and Discussion

7.1 Screenshots of User Interface (UI) for the respective module

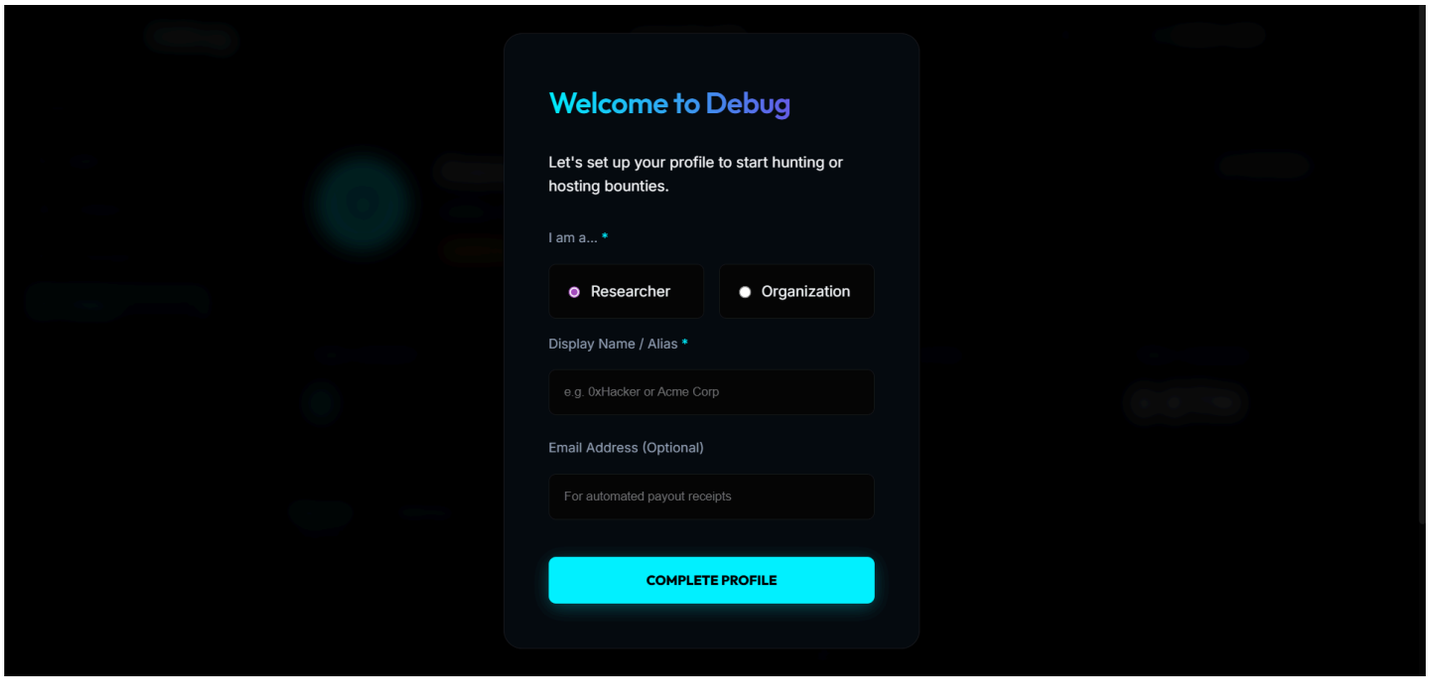

Fig 7.1.1 – Login page

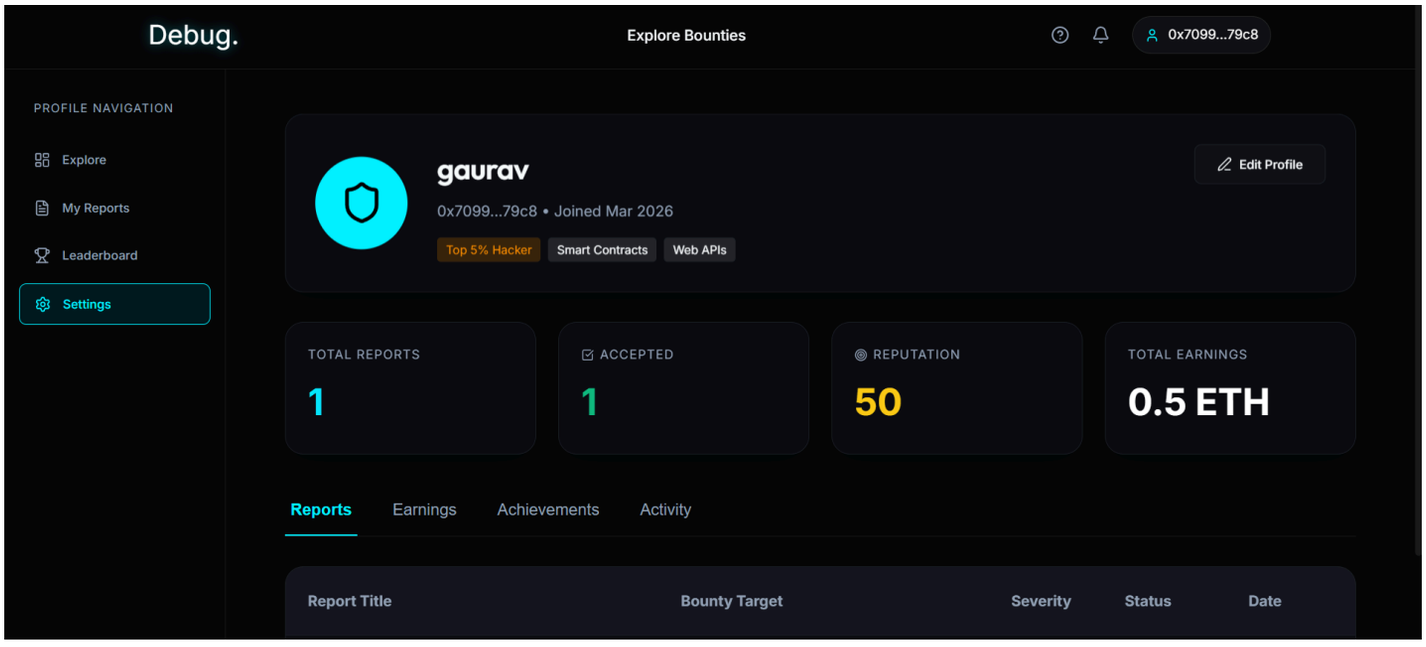

Fig 7.1.2 – Researcher Dashboard

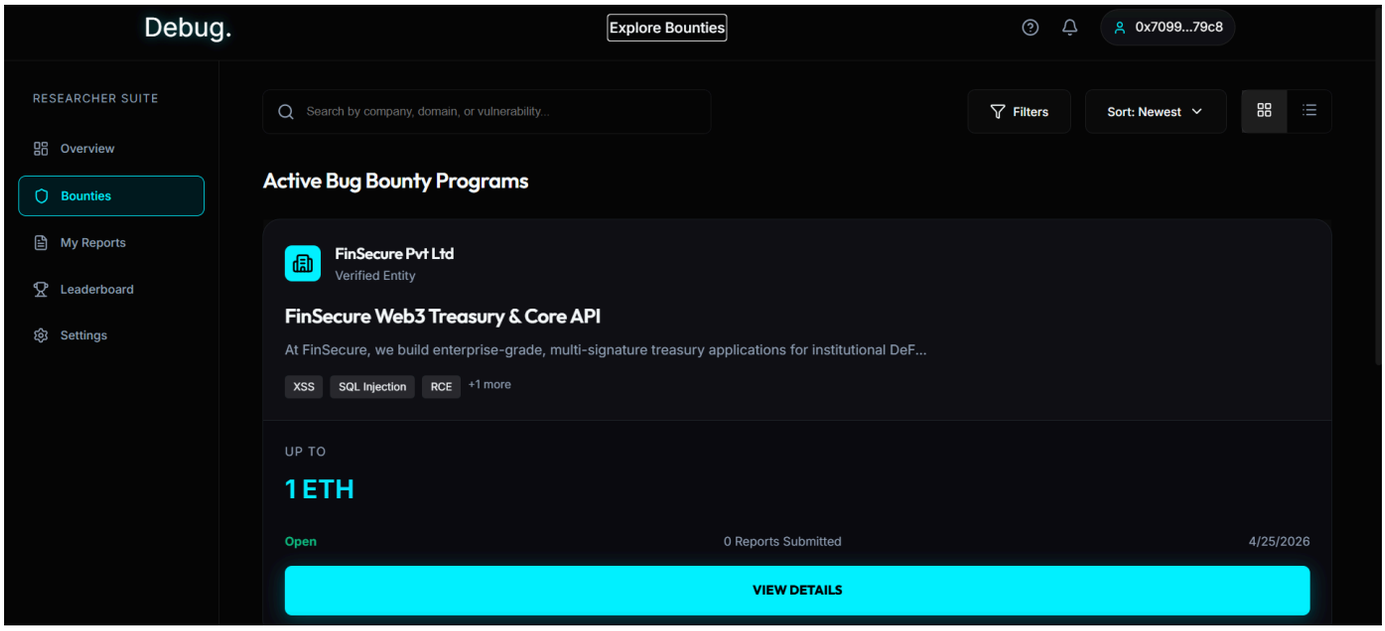

Fig 7.1.3 – Explore Bounties page

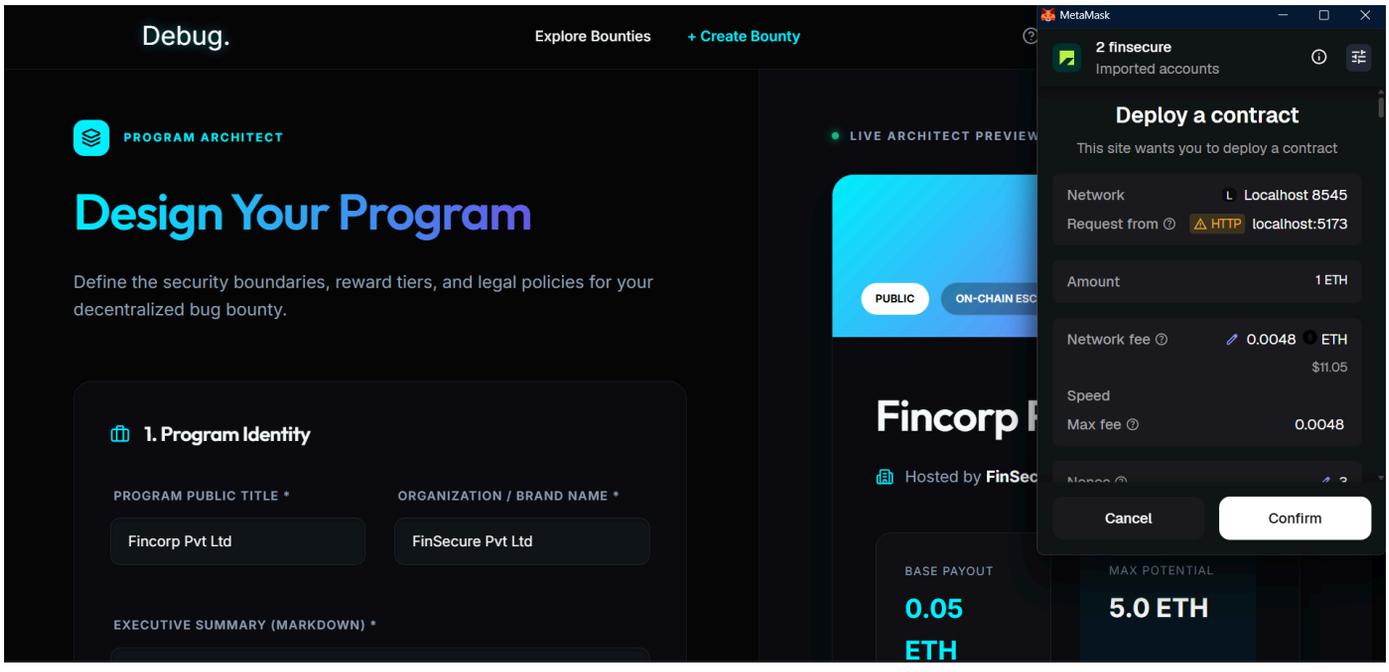

Fig 7.1.4 – Deploy Bounty Page

Fig 7.1.5 – Bounty Details Page

7.2 Performance Evaluation measures

Network	Type	Avg Gas Price	Cost (Unoptimized)	Cost (Optimized)	Savings
Hardhat Local	Dev Node	1 gwei	0.0014 ETH ($0.00)	0.0008 ETH ($0.00)	$0.00
Ethereum Sepolia	L1 Testnet	15 gwei	0.0217 ETH (~$65.25)*	0.0122 ETH (~$36.60)*	**~$28.65**
Polygon PoS	L2 Sidechain	30 gwei	0.0435 MATIC ($0.03)	0.0244 MATIC ($0.01)	**~$0.02**
zkSync Sepolia	ZK Rollup	0.1 gwei	0.0001 ETH ($0.43)*	0.00008 ETH ($0.24)*	**~$0.19**

Table no. 3 – Network Cost Comparison (Deployment Cost)

The performance of our Debug contracts is greatly influenced by gas efficiency. As shown in the table above, our code base is able to yield consistent savings of about ~43% in deployment gas cost for all tested networks.

Some key features used to achieve gas optimization are -

- **Variable packing:** Combining smaller variables together into a single 256-bit slot to reduce costly state changes.
- **Custom Errors:** Using parameter-less custom error instead of expensive require() statements to save on contract bytecode space.
- **Data Structures:** Mappings instead of arrays allow for efficient, $O(1)$ constant-time access to data.
- **Implications for Network Feasibility:** From our findings, we must choose an appropriate Layer-2 solution to scale Debug. An Ethereum L1 deployment using the testnet Sepolia incurs an exorbitant cost of about $36.60 which causes huge friction for corporations.

However, utilizing Layer-2 solutions such as Polygon PoS and zkSync allows us to reduce transaction fees drastically to cents per transaction ($0.01-$0.24). Thus, although Ethereum offers security, Debug must run natively on Layer-2 environments to be economically feasible and scalable.

7.3 Input Parameters / Features considered

Debug itself runs as a hybrid decentralized application, making the efficiency of Debug largely dependent upon the ability of processing different kinds of input data over blockchain networks, external AI models, and conventional database structures.

Some important parameters being evaluated by Debug include the following:

1. Web3 Cryptographic Input and Escrow Parameters The fundamental basis of trust on which the Debug network functions is based on off-chain and on-chain cryptographic inputs taking place in lieu of Web2 logins.

2. Wallet Address signatures: The user authentication process is solely dependent on the validation of signatures generated using a Web3 wallet (e.g., MetaMask).

3. Threat Intelligence Submissions (Data by Researcher) Certain raw data from the vulnerability submission process is extracted and evaluated.

4. PoC (Proof-of-Concept) Architecture: Command raw data, code snippets, and exploit execution steps written in Markdown. Front-end interprets this data for display, whereas the back-end uses this data to power AI analyses.

5. Threat Claims Metrics: The severity scale based on the CVSS scoring system chosen by the threat researcher by hand. This will serve as the base line in comparison with algorithm-based evaluations.

6. Bounds for Generative AI Triaging (AI Triage Input): For automated vulnerability triage with maximum efficiency, highly structured data are inputted into the Gemini AI Engine.

7.4 Comparison of results with existing systems

Comparing the benefits of Debug to those of conventional bug bounty platforms on the Web2 market shows several clear strengths in three core categories.

- Speed of Triage: Traditional bug bounty platforms like HackerOne and Bugcrowd have to go through lengthy manual review processes for each new submission that enters the platform. Consequently, hackers end up waiting days or even weeks for the first response. Debug utilizes the Flash generation model of Gemini 1.5 that analyzes each incoming report instantly, giving it a triage status immediately and eliminating any delays associated with traditional methods.

- Guaranteed Payouts: The major problem with traditional bug bounty platforms is that they serve as centralized custodians of the bounty money, exposing payout processes to numerous bottlenecks, including platform policies, bank transfers, and internal delays. Debug uses a Solidity smart contract as its trustless payment processor, ensuring that once a hack bounty is approved, the transaction goes directly to the hacker's crypto wallet instantly.

- Fee Model: Web2-based companies generally take a commission of between 15% and 30% on all bounty rewards, taking advantage of the fact that they are the only credible intermediaries. Debug solves the problem by enabling a direct connection between the companies and researchers using smart contracts, which means that only negligible Ethereum transaction fees are required, making sure researchers keep more of their pay.

7.5 Inference drawn

The results obtained from the project prove that combining blockchain technology and generative AI has the potential to significantly improve the problems of inefficiency that have been plaguing the classic bug bounty system for so long.

Using smart contract escrow changes the basis of the trust relationship between the organization and the researcher. Instead of assuming that the platform operates in their best interest, both sides deal with an immutable, determinate and publicly verifiable contract. The conditions under which the contract will be signed, the logic under which the research work will be approved, and the criteria for releasing funds – all this will be embedded into the contract itself, and neither side will be able to change these terms after the bounty is made.

Chapter 8: Conclusion

8.1 Limitations

Despite all its transparency and trustlessness in the bug bounty ecosystem, DeBug has some practical and technical limitations.

- **Volatility of Gas Fee -**
  While the system optimizes its smart contracts, resulting in lower costs for transactions, it is still subject to sudden increase in gas fees as a result of the network congestion.

- **AI Hallucination Risk -**
  As the platform employs Large Language Models for filtering, there could possibly be an instance where such a model would inaccurately output the information from the description of vulnerabilities or provide wrong estimation of CVSS, which requires approval by the organization before the reward is paid out.

- **Cryptographic Adoption Barrier -**
  Given that there still exists a considerable number of employees who deal with vulnerabilities and bugs who rely on the traditional fiat currency system, requiring them to engage in the processes involving cryptocurrencies will make for an adoption issue.

- **Off-Chain Dependency -**
  As sensitive information cannot be stored on blockchains, off-chain solutions need to be used for managing the reports.

- **Network Dependency -**
  The system depends on blockchain services, integration with wallets, and usage of external APIs of AI, and network outages can disrupt its operation.

8.2 Conclusion

DeBug completely reimagines the archaic and centralized structures of power that exist within contemporary bug bounty programs. With the use of the Bounty Escrow Smart Contract to control all financial transactions, security professionals are ensured a fair payment without any corporate delay or intermediation.

Moreover, organizations gain much from automating hours-long triage through the rapid analysis of reports via cutting-edge Generative AI algorithms. Tools like report deduplication, attack vector summary, and severity verification make this an efficient tool for streamlining operations. In sum, the integration of blockchain technology with artificial intelligence provides a balanced environment for security experts and organizations alike. Transparency, secure payments, and intelligent report analysis ensure greater assurance for bug bounties. Overall, the project lays out an excellent framework for a decentralized cybersecurity verification platform in the modern world.

8.3 Future Scope

There is significant room for improvement for this project as the demand for decentralized technology and cybersecurity is on the rise.

Multi-Signature Smart Contract Arbitration: Currently, the escrow contract is being used where the dispute could be resolved by neutral third-party individuals. However, in the future, the contract could be designed in a way that involves multiple parties who have the authority to approve transactions and arbitrate between organizations and researchers.

Cross-Blockchain Implementation: Currently, the DeBug is working on EVM-based blockchains; however, in the future, this solution could be extended to multiple blockchains. Thus, this will give organizations an option to start bounty programs in multiple chain ecosystems with higher efficiency and at reduced costs.

AI-Driven Vulnerability Response System: Instead of only triaging the issues, the AI-driven module of DeBug could integrate with CI/CD tools to provide developers with patches and testing requirements.

In summary, the future of DeBug is about extending its services from centralized to decentralized systems.

References

[1] V. Buterin, “A Next-Generation Smart Contract and Decentralized Application Platform,” *Ethereum Foundation Whitepaper*, vol. 3, no. 37, pp. 1–36, 2014.

[2] L. Luu, D. H. Chu, H. Olickel, P. Saxena, and A. Hobor, “Making smart contracts smarter,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 254–269.

[3] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” in *Proc. IEEE Int. Conf. on Decentralized Technology*, 2008, pp. 1–9.

[4] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, pp. 4171–4186.

[5] OpenAI, “GPT-4 Technical Report,” in *Proc. AI Alignment Conference*, 2023, pp. 1–55.

[6] Google DeepMind, “Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context,” *Google DeepMind Research Report*, 2024.

[7] D. Boneh, “Decentralized escrow systems utilizing blockchain protocols,” U.S. Patent 9,845,231, Oct. 15, 2021.

[8] FIRST.Org, “Common Vulnerability Scoring System v3.1: Specification Document,” Forum of Incident Response and Security Teams, Jun. 2019.

[9] MITRE Corporation, “Common Vulnerabilities and Exposures (CVE),” MITRE Database, 2024.

[10] G. Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger,” Ethereum Project Yellow Paper, 2014.

[11] MetaMask, “MetaMask Developer Documentation,” Consensys, 2024.

[12] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018. (Optional if AI theory mentioned)

[13] Ethers.js, “Ethers.js Documentation for Ethereum Wallet and Smart Contract Interaction,” 2024.

[14] Supabase Inc., “Supabase Documentation: Open Source Firebase Alternative,” 2024.

[15] Google, “Google Generative AI SDK Documentation,” Google AI Developers, 2024

Appendix - I

Inhouse/ ~~Industry~~ ~~Innovation/Research~~:

Class: D17 ~~A/B~~/C

Sustainable Goal: 8 & 9

Project Evaluation Sheet 2025 - 26

Group No.: 19

Title of Project: DeBug : A Decentralised Bug-Bounty Platform

Group Members: Rohit Shahi [D17C/58]; Gopal Vaijapurkar [D17C/66]; Gaurav Mahadeshwar [D17C/43]; Umesh Talaviya [D17C/64]

Engineering Concepts & Knowledge (5)	Interpretation of Problem & Analysis (5)	Design / Prototype (5)	Interpretation of Data & Dataset (3)	Modern Tool Usage (5)	Societal Benefit, Safety Consideration (2)	Environment Friendly (2)	Ethics (2)	Team work (2)	Presentation Skills (2)	Applied Engg&Mgmt principles (3)	Life - long learning (3)	Professional Skills (3)	Innovative Approach (3)	Research Paper (5)	Total Marks (50)
3	3	2	1	4	2	2	2	2	2	2	2	2	2	3	34

Comments: ① Many points in synopsis pending for implementation
② Teamwork ~~is lacking~~ need to improve

Name & Signature Reviewer1

Engineering Concepts & Knowledge (5)	Interpretation of Problem & Analysis (5)	Design / Prototype (5)	Interpretation of Data & Dataset (3)	Modern Tool Usage (5)	Societal Benefit, Safety Consideration (2)	Environment Friendly (2)	Ethics (2)	Team work (2)	Presentation Skills (2)	Applied Engg&Mgmt principles (3)	Life - long learning (3)	Professional Skills (3)	Innovative Approach (3)	Research Paper (5)	Total Marks (50)
3	2	2	2	3	2	2	2	1	1	2	2	2	1	3	31

Comments: 1) GEN-AI part pending
2) Not shown the actual working demonstration

Date: 5th March, 2026

Mrs. Yugchhaya Galphat

Name & Signature Reviewer 2

Inhouse/ ~~Industry~~
~~Innovation/Resear~~

Project Evaluation Sheet 2025 - 26

Class: D17 ~~A/B~~/C

Group No.: 19

Title of Project: DeBug : A Decentralised Bug-Bounty Platform

Group Members: Rohit Shahi [D17C/58]; Gopal Vaijapurkar [D17C/66]; Gaurav Mahadeshwar [D17C/43]; Umesh Talaviya [D17C/64]

Engineering Concepts & Knowledge (5)	Interpretation of Problem & Analysis (5)	Design / Prototype (5)	Interpretation of Data & Dataset (3)	Modern Tool Usage (5)	Societal Benefit, Safety Consideration (2)	Environment Friendly (2)	Ethics (2)	Team work (2)	Presentation Skills (2)	Applied Engg&Mgmt principles (3)	Life - long learning (3)	Professional Skills (3)	Innovative Approach (3)	Research Paper (5)	Total Marks (50)
4	4	3	2	4	2	2	2	2	2	3	2	3	3	-	38

Comments: In-depth work is missing. Draft of research paper with more analysis must be ready before next review. Increase mentor interaction frequency & update progress in regular basis

Name & Signature Reviewer1

Engineering Concepts & Knowledge (5)	Interpretation of Problem & Analysis (5)	Design / Prototype (5)	Interpretation of Data & Dataset (3)	Modern Tool Usage (5)	Societal Benefit, Safety Consideration (2)	Environment Friendly (2)	Ethics (2)	Team work (2)	Presentation Skills (2)	Applied Engg&Mgmt principles (3)	Life - long learning (3)	Professional Skills (3)	Innovative Approach (3)	Research Paper (5)	Total Marks (50)
4	4	3	2	4	2	2	2	2	2	3	2	3	3	-	38

Comments: 1) Research paper draft is missing
2) Scope of project need to be Refined

Date: 7th February, 2026

Jugchhaya Galphat

Name & Signature Reviewer 2